

“초등부 1번. 크림빵” 문제 풀이

작성자: 박선재

부분문제 1

전체 크림빵들이 한 묶음을 이룬다. 따라서, 전체 크림빵 중에서 크림이 들어 있지 않은 빵들의 개수를 센다. 그 개수가 P 보다 작으면 1, 아니면 0을 출력하면 된다.

부분문제 2

각각의 크림빵들이 한 묶음을 이룬다. 따라서, 크림빵에 크림이 들어 있으면 그 크림빵을 팔 수 있고, 아니면 팔 수 없다. 따라서, 크림이 들어 있는 빵의 개수를 출력하면 된다.

부분문제 3

모든 크림빵에 크림이 들어있거나, 크림이 들어있지 않다. 따라서, 아무 빵이나 조사해서 크림이 들어 있으면 N 개 묶음 전부 팔 수 있다. 이때는 N 을 출력한다. 아니면 전부 팔 수 없으므로 0을 출력하면 된다.

부분문제 4

각 묶음에 속한 크림빵들에 모두 크림이 들어있거나, 들어 있지 않다. 따라서, 각 크림빵 묶음에 대해 아무 크림빵이나 조사해서, 크림이 들어있으면 그 묶음을 팔 수 있고, 아니면 팔 수 없다. 팔 수 있는 묶음의 개수를 세어서 출력하면 된다.

부분문제 5

$P = 1$ 이기 때문에, 각 묶음에 대해 크림이 없는 빵이 하나라도 있으면 그 묶음을 팔 수 없다. 각 묶음 내에 크림이 없는 빵이 하나라도 있는지 조사한다. 하나라도 있다면 그 묶음은 팔 수 없고, 아니라면 팔 수 있다. 팔 수 있는 묶음의 개수를 세어서 출력하면 된다.

부분문제 6

부분문제 5에서 했던 것처럼, 각 묶음에 대해 모든 빵을 조사하되, 크림이 없는 빵의 개수를 세어서 저장한다. 그 후 크림이 없는 빵의 개수를 P 와 비교한다. 크림이 없는 빵의 개수가 P 보다 작으면 그 묶음은 팔 수 있고, 아니면 팔 수 없다. 크림이 없는 빵의 개수를 세었던 방식과 비슷하게, 팔 수 있는 묶음의 개수를 저장한다. 모든 묶음에 대해 조사가 끝나면, 팔 수 있는 묶음의 개수를 출력한다.

“초등부 2번 / 고등부 1번. 대피소” 문제 풀이

작성자: 오주원

부분문제 1

대피소를 설치하지 않을 집을 선택한 경우, 해당 집과 가장 가까운 집 사이의 거리가 대피소로 이동하기 위한 최대 거리가 된다. 따라서, 문제의 정답을 찾기 위해서는 각 집마다 가장 가까운 집과의 거리를 계산하고, 이 중에서 최솟값을 찾아야 한다. 즉, $\frac{N(N-1)}{2}$ 개의 거리 쌍 중에서 가장 작은 거리가 문제의 정답이 된다.

부분문제 2

어떤 집에 대피소를 설치하더라도 가장 먼 집은 1번 집 또는 N 번 집이므로 대피소와 두 집 사이의 거리만 고려하면 된다. 대피소를 K 번 집에 설치하면 최대 거리는 $N - K$ 와 $K - 1$ 중 더 큰 값이 된다. 찾아야 하는 것은 이 최대 거리를 최소화하는 K 이다. 중앙에 가까운 집을 선택할수록 최대 거리가 줄어드는 것을 알 수 있고, 따라서 가장 중앙에 가까운 집을 선택했을 때의 거리가 최소가 되어 $\lfloor \frac{N}{2} \rfloor$ 가 문제의 정답이 된다.

부분문제 3

대피소를 설치할 집 하나를 선택한 다음, 설치하지 않은 다른 집들과의 거리를 비교하여 가장 큰 값을 찾는다. 총 N 개의 집을 선택할 수 있고, 최대 거리를 구하는 데에 $O(N)$ 의 시간이 걸리므로 $O(N^2)$ 에 정답을 구할 수 있다.

부분문제 4

대피소를 설치할 집 하나를 선택하고, 설치하지 않은 집 중에서 하나를 골라 대피소를 설치한다. 그다음, 각 집과 두 대피소와의 거리를 비교해서 더 작은 값으로 정답을 갱신해준다. N 개의 집 중에 2개의 집을 선택하는 방법은 $\frac{N(N-1)}{2}$ 개가 있고, 거리를 구하는 데 $O(N)$ 의 시간이 소요된다. 따라서, $O(N^3)$ 의 시간복잡도로 정답을 구할 수 있다.

부분문제 5

대피소를 1번 집과 N 번집에 설치하면 최대 거리는 $X_N - X_1$ 보다 작아진다는 것을 알 수 있다. 따라서 0부터 $X_N - X_1$ 까지 1씩 증가시키며 하나씩 정답으로 가능한지 확인한 이후, 가능하면 그 값을 바로 출력해주고 프로그램을 종료하는 것으로 항상 정답을 찾을 수 있다.

정답으로 A 가 가능한지 확인하기 위해서 탐욕적인 방법을 사용한다. 가장 작은 번호의 집에 설치할 대피소는 1번 집과의 거리가 A 이하여야 하고, 그중 가장 번호가 큰 집을 선택해야 그 대피소와 거리가 A 이하인 집의 개수가 가장 많아진다. 다음으로 중간에 설치할 대피소는 첫 번째로 설치한 대피소와 거리가 A 보다 크면서 번호가 가장 작은 집을 z 라고 할 때, z 와의 거리가 A 이하여야 한다. 앞과 마찬가지로 이유로, z 와 거리가 A 이하이면서 가장 번호가 큰 집에 대피소로 설치해야 하고, 같은 방법으로 세 번째 대피소를 설치할 집도 찾아주면 된다. 이렇게 대피소를 설치하면 1번 집부터 세 번째 대피소를 설치한 집까지는 모두 대피소와의 거리가 A 이하가 되며, 세 번째 대피소를 설치한 집보다 번호가 크면서 가장 멀리 떨어진 집은 N 번 집이므로 두 집 사이의 거리를 구하여 모든 거리가 A 이하인지 확인할 수 있다. 이때, 모든 거리가 A 이하라면 A 는 정답이 될 수 있고, 없다면 A 는 정답이 될 수 없다.

1부터 X_N 까지 각 값을 가능한지 확인하는데 $O(K + N)$ 의 시간이 걸리므로 $O((K + N)X_N)$ 에 문제를 해결할 수 있다. 추가로, 이분 탐색을 이용해 $O(K \log X_N \log N)$ 에 문제를 해결할 수도 있다.

부분문제 6

대피소를 설치할 세 집을 선택하고, 선택한 세 대피소와 설치하지 않은 집 간의 거리를 비교하여 가장 작은 값의 최댓값이 정답의 후보가 된다. N 개의 집 중에 3개의 집을 선택하는 방법은 $\frac{N(N-1)(N-2)}{6}$ 개가 있으며, 거리를 구하는 데 $O(N)$ 의 시간이 걸리므로 $O(N^4)$ 에 정답을 구할 수 있다.

“초등부 3번. 아이템 획득” 문제 풀이

작성자: 나정휘

부분문제 1

좌표의 크기가 작기 때문에 지도 전체를 이차원 배열로 저장할 수 있다. (x, y) 를 지날 때 얻을 수 있는 아이템의 개수를 관리하는 배열 $A(x, y)$ 를 만든 다음, 자동차가 이동할 때마다 지나가는 위치의 값을 모두 더하면 정답을 구할 수 있다.

부분문제 2

N 과 Q 가 작으므로 자동차가 이동할 때마다 모든 상자를 보면서, 각 상자를 지나는지 확인하면 $O(NQ)$ 시간에 정답을 구할 수 있다.

부분문제 3

모든 상자의 x, y 좌표가 모두 다르기 때문에 이동할 때마다 최대 1개의 상자만 지난다.

각각의 x, y 좌표마다 유일하게 존재하는 상자의 위치와 아이템의 개수를 저장하면, 이동할 때마다 상수 시간에 상자를 지나는지 알 수 있다.

부분문제 4

모든 상자에 아이템이 1개만 존재하므로, 이동할 때마다 지나는 상자의 개수만 구해도 된다.

모든 이동은 x 축 또는 y 축과 평행하다는 점을 이용해서, 각 x, y 좌표에 존재하는 상자를 모아서 저장하자. 즉, $X_i = x$ 좌표가 i 인 상자들의 y 좌표를 저장한 배열, $Y_i = y$ 좌표가 i 인 상자들의 x 좌표를 저장한 배열을 만든다.

이동할 때마다 지나는 상자의 개수를 구하는 것은 배열에서 l 이상 r 이하인 수의 개수를 구하는 것으로 생각할 수 있다. 이런 문제는 배열을 정렬한 뒤, 이분 탐색을 이용해 l 보다 크거나 같은 첫 번째 위치, 그리고 r 보다 큰 첫 번째 위치를 구하는 것으로 해결할 수 있다.

모든 원소를 x, y 좌표에 따라 분류해서 정렬하는데 $O(N \log N)$ 시간이 걸리고, 이동할 때마다 길이가 최대 N 인 배열에서 이분 탐색을 2번 수행하므로 각 이동을 처리하는데 $O(\log N)$ 시간이 걸린다. 따라서 부분문제 4를 $O((N + Q) \log N)$ 시간에 해결할 수 있다.

부분문제 5

부분문제 4에서는 구간에 포함된 상자의 개수만 구하면 됐지만, 부분문제 5를 해결하기 위해서는 아이템의 개수의 합을 구해야 한다.

부분문제 4에서 했던 것처럼 X_i, Y_i 배열을 관리하는데, 이때 상자의 좌표와 아이템의 개수를 함께 저장하자. 배열에서 구간의 합을 빠르게 구할 수 있어야 한다. 누적 합 배열을 이용하면 상수 시간에 구할 수 있으므로, 배열을 좌표 오름차순으로 정렬한 다음 아이템의 개수에 대한 누적 합 배열을 만들면 부분문제 5도 $O((N + Q) \log N)$ 에 해결할 수 있다.